PRODUCT
REQUIREMENTS
DOCUMENT

Skeleton-Based Industrial AI Pipeline

Free-data-first approach to building a working skeleton extraction, auto-annotation, and action recognition system — before a single device is manufactured.

PROJECT
Tentellect / Data Infra

AUTHOR
Anvin (AI Lead)

VERSION
1.0

STATUS
Active / Draft

DATE	HORIZON
April 2025	8 Weeks

00 Context & Strategic Reframe

The original plan assumed hardware-first: build a device, deploy to a factory, collect data, then train models. This PRD inverts that order. The insight is simple — **you cannot pitch to robotics companies with a diagram. You pitch with a running system.**

This document defines the pipeline required to go from zero to a working, demo-ready skeleton AI system using entirely free public datasets. The device and factory deployment come

after you have traction to show.

STRATEGIC GOAL

Build a complete skeleton extraction, auto-annotation, and action recognition pipeline on free public industrial data. Use this as the technical proof to unlock access to robotics companies' proprietary datasets and factory deployments — without spending money on hardware or factory negotiations first.

KEY CONSTRAINT

Two engineers. No dedicated GPU cluster. No factory access yet. Every

> architectural decision must reflect this reality. Colab Pro ($10/month) and Vast.ai spot instances ($0.30/hr) are the compute budget. Everything else is free.

01 Goal & Success Definition

By end of Week 8, the system must be able to accept **any industrial video or image folder as input** and return:

- A COCO-format keypoints JSON with 33-landmark skeleton per detected worker per frame
- Per-keypoint confidence scores with automatic

accept/review/discard classification

- PPE compliance score per worker (helmet, vest, gloves detected relative to skeleton)
- A 20-dimension feature vector per worker per 1-second window (spatial + temporal)
- Action class prediction (fall / bend / walk / reach / idle / danger posture)
- Risk score ∈ [0.0, 1.0] per worker per window
- Auto-annotation export in COCO JSON, YOLO TXT, and CSV formats
- A Label Studio review queue for all frames below the confidence threshold

> **THE DEMO DELIVERABLE**
>
> The output of Week 8 is a single script: `python run_pipeline.py`

`--input /path/to/video` that processes any factory video and produces annotated skeletons, risk scores, and a dataset export — in under 5 minutes for a 1-hour video. This is what you show robotics companies.

02 Free Datasets — What to Download

All datasets below are free, publicly available, and legally usable for training ML models. Listed in priority order — download Tier 1 in Week 1, Tier 2 when pipeline is running.

Tier 1 — Start Immediately

DATASET	SIZ
COCO 2017 Keypoints	~18
SH17 Dataset	~2.
Roboflow Worker Safety	~30
iSafetyBench	~8G

Tier 2 — Download When Pipeline is Running (Week 3+)

DATASET	SIZE
OpenMarcie	~400
Human3.6M	~320
NTU RGB+D 120	~130
CMU Panoptic Studio	Vari

AGENT TASK — DATASET DOWNLOAD

Write a `scripts/download_datasets.sh` script that: (1) downloads COCO 2017 train+val keypoints zip from cocodataset.org and extracts to `data/coco/`, (2) clones the SH17 dataset repo from `github.com/ahmadmughees/SH17dataset` into `data/sh17/`, (3) downloads the iSafetyBench videos from the arXiv paper link into `data/isafety/`, (4) downloads the Roboflow Worker Safety dataset via the Roboflow API (free tier). Each download should verify checksum and print file counts on completion. Include a `requirements.txt`

> entry for `roboflow` CLI.

03 Download & Setup Instructions

Repository Structure

```
EXPECTED DIRECTORY TREE AFTER SETUP

tentellect-pipeline/ ├── data/ │ ├── coco/ │ │ ├── images/train2017/ # 118k images │ │ ├── images/val2017/ # 5k images │ │ └── annotations/ │ │ ├── person_keypoints_ │ │ └── person_keypoints_ │ ├── sh17/ │ ├── images/ # 8,099 industrial images │ │ └── labels/ # YOLO format annotations │
```

```
├── isafety/
│ │ ├── clips/ # 1,100 video clips │ │ └── annotations.json
│ ├── openmarcie/ # Week 3+ after request approved │ └── processed/ # Pipeline outputs go here │ ├── skeletons/ # Per-session COCO keypoint JSON │ ├── features/ # 20-dim feature vectors (.npy) │ └── review_queue/ # Low-confidence frames for Label Studio
├── src/ │ ├── ingestion/ │ ├── skeleton/ │ ├── imu/ │ ├── annotation/ │ ├── features/ │ ├── training/ │ └── serving/
├── scripts/ │ ├──
```

```
download_datasets
| ├──
validate_datasets
| └──
run_pipeline.py
├── models/ |
└──
checkpoints/
├── configs/
| └──
pipeline.yaml
├── tests/
└── README.md
```

Environment Setup

```
ENVIRONMENT —
PYTHON 3.11 +
CUDA 12.1

# Create
environment
conda create
-n tentellect
python=3.11 -
y conda
activate
tentellect #
Core
inference pip
install
ultralytics==8.2.
# YOLOv8-Pose
pip install
mediapipe==0.10.1
# 33-landmark
extraction
pip install
onnxruntime-
gpu==1.18.0 #
```

```
ONNX
inference
backend #
Data &
annotation
pip install
pycocotools==2.0.
# COCO format
parsing pip
install
label-
studio==1.12.0
# Local
annotation UI
pip install
label-studio-
sdk # Python
integration
pip install
dvc[s3]==3.50.0
# Dataset
versioning #
ML stack pip
install
torch==2.3.0
torchvision #
Training
backbone pip
install
xgboost==2.0.3
# Risk scorer
pip install
shap==0.45.0
# Decision
explanation
pip install
mlflow==2.13.0
# Experiment
tracking pip
install
optuna==3.6.0
#
Hyperparameter
```

```
search #
Utilities pip
install
opencv-
python-
headless #
Frame
processing
pip install
numpy scipy
pandas #
Feature
computation
pip install
tqdm rich
pyyaml # CLI
tooling pip
install
fastapi
uvicorn #
Serving layer
```

COMPUTE NOTE

Run pipeline on **Vast.ai RTX 4090 spot instance (~$0.45/hr)** for dataset processing. Use **Google Colab Pro ($10/month)** for model training experiments. Local dev machine handles code, configs, and

> data validation only. Terminate cloud instances immediately after each run — never leave idle.

04 Data Quality Gates

Every frame entering the pipeline passes through three gates. No frame that fails all three gates enters the training dataset.

GATE	CONDI
G1 — Detection	At lea perso detec bbox confi
G2 — Pose Confidence	Mean confi acros critic

GATE	CONDI
G3 — Keypoint Plausibility	Anat valid — lin ratio ±30% biom norm self- inter skele

AGENT TASK — QUALITY GATE IMPLEMENTATION

Implement `src/annotation/quality_gates.py` with a `QualityGate` class that takes a frame's keypoint dict (COCO format) and returns one of `AUTO_ACCEPT`, `REVIEW`, or `DISCARD` with a reason string. The plausibility check must validate: (1) shoulder width / height ratio in [0.2, 0.8], (2) arm length symmetry within 20%, (3) no keypoint coordinate

outside frame bounds, (4) minimum 8 of 17 keypoints visible. Write pytest tests covering edge cases: fully occluded worker, multiple workers, headless detection.

05 Pipeline Architecture

```
END-TO-END PIPELINE — DATA FLOW

INPUT (video file / image folder / RTSP stream) | ▼
┌──────────────────
| INGESTION LAYER | | ffmpeg frame extraction @ 2fps (dataset) / 15fps (RT) |
| Resolution normalize → 640×480 | | CLAHE
```

```
preprocessing
(factory
lighting
compensation)
| | Metadata
tagging:
source_id,
frame_idx,
timestamp |
└──────────────
| ▼
┌──────────────
| SKELETON
EXTRACTION |
| Stage 1:
YOLOv8-Pose →
person
detection +
17 COCO kps |
| Stage 2:
MediaPipe
Pose → 33
landmarks +
world 3D | |
Stage 3:
ByteTrack →
persistent
worker IDs
across time |
| Stage 4:
DeepSORT Re-
ID → cross-
camera
identity | |
Output:
{track_id,
keypoints_17,
keypoints_33,
| |
confidence[],
bbox,
world_3d[]} |
└──────────────
```

```
    | ▼
    ┌──────────────────────────────
    | PPE
    CLASSIFIER
    (parallel
    stream) | |
    EfficientNet-
    Lite0 on
    cropped
    worker ROI |
    | Classes:
    helmet, vest,
    gloves,
    glasses
    (multi-label)
    | | Output:
    ppe_score ∈
    [0,1] per
    class |
    └──────────────────────────────
    | ▼
    ┌──────────────────────────────
    | QUALITY
    GATES (G1,
    G2, G3) | | →
    AUTO_ACCEPT |
    REVIEW |
    DISCARD |
    └──────────────────────────────
    |
    ┌──────────┴──────────┐
    ▼ ▼
    AUTO_ACCEPT
    REVIEW | |
    Feature
    Extract Label
    Studio |
    review queue
    ▼
    ┌──────────────────────────────
    | FEATURE
    EXTRACTION |
    | Spatial:
```

```
angles,
zone_id,
ppe_score,
bbox_area | |
Temporal:
velocity,
accel,
dwell_time
(30-frame
window) | |
IMU fusion:
torso_pitch,
roll,
fall_flag (if
available) |
| Output:
float32[20]
per worker
per 1-second
window |
└──────────────
| ▼
┌──────────────
| RISK
SCORING | |
XGBoost
classifier →
risk_score ∈
[0.0, 1.0] |
| SHAP
explanation →
top 5
contributing
features | |
Action class
→ fall / bend
/ walk /
reach / idle
|
└──────────────
| ▼
┌──────────────
| STORAGE &
```

```
EXPORT | |
SQLite:
annotations +
metadata | |
COCO JSON
export
(keypoints
format) | |
YOLO TXT
export (for
fine-tuning)
| | CSV
export
(features +
risk scores)
| | DVC
commit:
dataset
version
locked |
└──────────────
```

06 Ingestion Layer

Accepts any video file (MP4, AVI, MKV), image folder, RTSP stream URL, or COCO JSON directory. Normalizes all inputs to a standard frame stream before skeleton extraction.

Frame Extraction Strategy

MODE	SOURCE
dataset	video file
realtime	RTSP / webcam
batch	image folder

Preprocessing — CLAHE for Factory Lighting

```
SRC/INGESTION/PREPR
— CORE
FUNCTION

import cv2
import numpy
as np def
preprocess_frame(
np.ndarray) -
> np.ndarray:
""" Apply
CLAHE
(Contrast
Limited
Adaptive
Histogram
Equalization)
```

```
to compensate
for uneven
factory floor
lighting.
Converts to
LAB,
equalizes L
channel only,
converts
back.
Dramatically
improves pose
estimation
under
fluorescent/shado
conditions.
""" lab =
cv2.cvtColor(fram
cv2.COLOR_BGR2LAB
l, a, b =
cv2.split(lab)
#
clipLimit=2.0
prevents
over-
amplification
of noise #
tileGridSize=
(8,8) means
64 local
regions per
frame clahe =
cv2.createCLAHE(c
tileGridSize=
(8,8)) l_eq =
clahe.apply(l)
merged =
cv2.merge([l_eq,
a, b]) return
cv2.cvtColor(merg
cv2.COLOR_LAB2BGR
def
normalize_resolut
```

```
    np.ndarray,
    target: tuple
    = (640, 480))
    ->
    np.ndarray:
    """Letterbox
    resize — no
    distortion,
    pads with
    grey."""  h, w
    =
    frame.shape[:2]
    scale =
    min(target[0]/w,
    target[1]/h)
    new_w, new_h
    =
    int(w*scale),
    int(h*scale)
    resized =
    cv2.resize(frame,
    (new_w,
    new_h))
    canvas =
    np.full((target[1
    target[0],
    3), 114,
    dtype=np.uint8)
    x0 =
    (target[0] -
    new_w) // 2
    y0 =
    (target[1] -
    new_h) // 2
    canvas[y0:y0+new_
    x0:x0+new_w]
    = resized
    return canvas
```

AGENT TASK — INGESTION LAYER

> Implement `src/ingestion/ingestor.py` with a `DataIngestor` class. It must accept `source` (path string or URL), `mode` (dataset / realtime / batch), and `session_id`. It must yield preprocessed frames as a generator with metadata dict attached: `{frame_idx, timestamp_ms, source_id, session_id}`. Use ffmpeg-python for video decoding, not OpenCV VideoCapture — ffmpeg is faster and handles more codecs. Write a CLI wrapper in `scripts/ingest.py` that takes a source path and prints frame count + sample metadata to stdout.

07 Skeleton Extraction

Two-model approach: YOLOv8-Pose for multi-person detection and 17 COCO keypoints, MediaPipe Pose for 33-landmark detail and world 3D coordinates. ByteTrack handles identity persistence across frames.

Model Selection Logic

CONDITION	PRIMA
1 person in frame	Medi
2+ persons in frame	YOLO
Low confidence (<0.7)	Both

CONDITION	PRIMA
Video sequence (not image)	YOLO

Skeleton Output Schema

```
OUTPUT JSON SCHEMA PER DETECTION — STORED IN SQLITE + COCO JSON

{
"frame_idx": 1420,
"timestamp_ms": 47333,
"session_id": "sh17_batch_001",
"track_id": "worker_003",
"bbox": [124, 88, 312, 450], # [x1, y1, x2, y2]
"detection_confid
0.94,
"keypoints_17": [ # COCO 17 keypoints
{"name":
```

```
"nose", "x":
218, "y":
102, "conf":
0.97},
{"name":
"left_shoulder",
"x": 167,
"y": 189,
"conf":
0.92}, # ...
15 more
keypoints ],
"keypoints_33":
[/* MediaPipe
33 landmarks
*/],
"world_3d": [
# MediaPipe
world
coordinates
(metres)
{"name":
"nose", "x":
0.02, "y":
-0.44, "z":
-0.08}, # ...
32 more world
landmarks ],
"mean_confidence"
0.88,
"quality_gate":
"AUTO_ACCEPT",
"ppe": {
"helmet":
0.94, "vest":
0.87,
"gloves":
0.21,
"glasses":
0.05 } }
```

AGENT TASK — SKELETON EXTRACTOR

Implement `src/skeleton/extractor.py` with a `SkeletonExtractor` class. Constructor takes `config: dict` with model paths, confidence thresholds, and device (cpu/cuda). The `extract(frame, metadata)` method returns a list of skeleton dicts matching the schema above. Implement the model selection logic table above — auto-detect person count from YOLO detection before deciding which model runs. ByteTrack must be initialized with `track_thresh=0.5, track_buffer=30, match_thresh=0.8`. Cache the MediaPipe Pose

> model in memory — do not reinitialize per frame. Run unit tests with pytest on 3 reference frames from SH17 dataset confirming non-empty keypoint output.

08 IMU Fusion Layer

When IMU data is available (OpenMarcie dataset has synchronized IMU, future field device will have it), this layer fuses accelerometer readings with skeleton keypoints to produce **occlusion-robust skeleton enrichment** and **physics-based auto-labels** (fall,

impact, rapid movement).

When IMU data is not available (COCO, SH17, iSafetyBench processing), this layer is skipped and IMU features are zeroed in the feature vector with an `imu_available: false` flag.

IMU Auto-Labels (Free Annotation from Physics)

LABEL
fall_event
impact_detected
rapid_movement
static_posture

LABEL
bending_forward

AGENT TASK — IMU FUSION MODULE

Implement `src/imu/fusion.py` with two classes: (1) `ComplementaryFilter` that takes ax/ay/az at variable sample rates and outputs pitch/roll in degrees, using alpha=0.98. (2) `IMUAutoLabeler` that takes a buffer of IMU readings (list of dicts with ax/ay/az/timestamp_ms) and returns a list of `AutoLabel` dataclasses with fields: label_type, confidence, start_ms,

end_ms, metadata. Must handle the case where IMU and video are on different sample rates — interpolate IMU to match video frame timestamps using numpy interp. Include a `test_imu_fusion.py` that generates synthetic fall signature data and verifies the fall_event label fires correctly.

09 Auto-annotation & Review System

The annotation system has two paths: auto-accepted frames go directly to dataset storage; review-flagged frames are pushed to

a local Label Studio instance for human correction. The goal is less than 10 minutes of human annotation per hour of processed video.

Label Studio Integration

1. **Run Label Studio locally**

   `label-studio start --port 8080` — runs entirely on your machine, no cloud account needed

2. **Pipeline pushes review tasks via SDK**

   When G2 returns REVIEW, the pipeline calls `label_studio_sdk` to create a task with the frame image + auto-skeleton overlay as the pre-annotation. Reviewer sees what the model got and just corrects it.

3. **Reviewer corrects keypoints**

(target: 30s per frame)

Pre-annotated skeletons mean reviewers drag existing points, not place them from scratch. 30s/frame is achievable when >80% of keypoints are already correct.

4 **Completed reviews pulled back via SDK and merged into dataset**

Reviewed annotations replace auto-predictions in SQLite and are flagged `human_verified: true`. These count double in training sample weighting.

AGENT TASK — ANNOTATION MODULE

Implement `src/annotation/annotator.py` with a `AnnotationRouter` class. It takes a skeleton dict

and quality gate result and routes to either: (1) `DatasetWriter` — writes to SQLite and appends to COCO JSON in-memory, (2) `LabelStudioPusher` — creates a Label Studio task with the frame image encoded as base64 and the skeleton as pre-annotation in Label Studio's keypoint format. The LabelStudio project must use the `KeypointLabels` template with all 17 COCO keypoints defined. Write a `scripts/pull_reviews.py` that fetches completed tasks from Label Studio and merges them back into the SQLite database, marking

records as `human_verified=True`.

10 Storage & Dataset Versioning

STORE	TECH
Annotations	SQLi
Frames	JPEG file
Feature vectors	NumP .npy
COCO export	JSON
Models	MLfl

DVC Dataset Versioning

```
SCRIPTS/COMMIT_DATA

#!/bin/bash # Run after each major dataset update # Tags the dataset with frame count + date for reproducibility
FRAME_COUNT=$(pyt -c "import sqlite3; c=sqlite3.connect print(c.execute('COUNT(*) FROM annotations WHERE quality_gate=\"AU [0])")
VERSION="v$(date +%Y%m%d)_${FRAME_
dvc add data/processed/
git add data/processed.dv .gitignore
git commit -m "dataset: ${VERSION} — auto-accepted frames + reviewed" git tag -a "$VERSION" -m "Dataset snapshot:
```

```
$FRAME_COUNT
frames" echo
"Committed
dataset
version:
$VERSION"
```

AGENT TASK — STORAGE LAYER

Implement `src/annotation/storage.py` with a `DatasetWriter` class. Schema: SQLite table `annotations` with columns: id (UUID), session_id, frame_idx, timestamp_ms, track_id, quality_gate, human_verified, skeleton_json (TEXT), ppe_json (TEXT), feature_vector (BLOB), risk_score (REAL), action_class (TEXT), created_at. Implement a `export_coco(output_path)` method that queries all

> AUTO_ACCEPT and human_verified=True records and writes a valid COCO keypoints JSON. Validate the export with `pycocotools` COCO loader — it must load without errors.

11 Training Loop

Three models are trained sequentially. Each model uses the output of the previous as a component. Training runs on Colab Pro / Vast.ai — not local machine.

Model 1 — Industrial Pose Detector (Fine-tune)

PARAMETER	VA
Base model	`y` po

PARAMETER	VA
	pr on
Fine-tune data	SH Ro W Sa (ir do
Epochs	50 st pa
Image size	64
Batch size	16 on
Augmentation	M HS ho fli ve fo
Target mAP50	> (in va
Export	ON Te IN fu de

Model 2 — Action Classifier

PARAMETER	VALU
Architecture	ST-C Tem Con Netv
Input	Seqı skel at 3(keyp
Training data	NTU (act iSaf clas
Output classes	fall, ben wall reac run, mac dan
Framework	PyT MM imp
Target accuracy	Top iSaf cate

Model 3 — Risk Scorer

PARAMETER	VA
Architecture	XG (gr bo — int

PARAMETER	VA
	fas ne
Input	20 fea pe pe wi
Output	ris [0.
Labels	De iSa ha an IM lab
Explainability	SH Tr — fea pr
Calibration	Isc reg EC 0.0
Inference	< 2 pr an

AGENT TASK — TRAINING SCRIPTS

Create three training scripts: (1) `src/training/train_pose.py` — uses

Ultralytics CLI wrapper to fine-tune YOLOv8s-pose on SH17+Roboflow data, logs all metrics to MLflow, exports ONNX on completion. (2) `src/training/train_action.py` — loads NTU RGB+D skeleton sequences, builds ST-GCN with MMAction2, trains with PyTorch Lightning, saves best checkpoint. (3) `src/training/train_risk.py` — loads feature vectors from `data/processed/features/`, trains XGBoost with 5-fold CV, calibrates with isotonic regression, computes and saves SHAP explainer. All three must log to the same MLflow experiment `tentellect_pipeline_v1`.

12 8-Week Phase Plan

PHASE 0 · WEEKS 1–2

Foundation & Data

Download all Tier 1 datasets. Set up repo structure and environment. Validate dataset integrity. Run baseline YOLOv8-Pose on 100 SH17 images to establish confidence distribution baseline.

PHASE 1 · WEEKS 3–4

Pipeline Core

Build ingestion, skeleton extractor, quality gates, and storage layer. Process all of SH17 through the pipeline. Target: 5,000+ auto-accepted labeled frames in dataset. Label Studio running locally.

PHASE 2 · WEEKS 5–6

IMU Fusion + Training

Download OpenMarcie, process 37hr of industrial video. Run IMU fusion on OpenMarcie wearable data. Fine-tune

PHASE 3 · WEEKS 7–8

Risk Scorer + Demo

Train XGBoost risk scorer. Build `run_pipeline.py` end-to-end script. Create a demo video processing any factory footage and returning annotated output. Measure all

YOLOv8-Pose on industrial images. Train action classifier on NTU RGB+D + iSafetyBench.

metrics against targets.

WEEK 1

Repo + Environment + Data Download

- Create GitHub repo with directory structure from §03
- Run `download_datasets.sh` — COCO, SH17, Roboflow Worker Safety, iSafetyBench
- Run `validate_datasets.py` — check frame counts, annotation formats, no corrupt files
- Set up conda environment, verify torch + ultralytics + mediapipe imports on dev machine
- Submit request forms for OpenMarcie + Human3.6M + NTU RGB+D (takes 3–7 days to approve)

WEEK 2

Baseline + Ingestion Layer

- Run baseline YOLOv8s-pose (pretrained) on 500 random SH17 images — record confidence distribution
- Build and test `DataIngestor` — process SH17 as batch mode, yield frames with metadata
- Build `preprocess_frame()` — verify CLAHE improves detection on dark factory images
- Measure: what % of SH17 frames pass G1 (person detected) with default model?

WEEK 3

Skeleton Extractor + Quality Gates

- Build `SkeletonExtractor` with model selection logic
- Build `QualityGate` with G1/G2/G3 — measure auto-accept rate on SH17

- Set up Label Studio locally, configure keypoint template, test push/pull cycle
- Process all 8,099 SH17 images — collect first dataset version

WEEK 4

Storage + Annotation + First Dataset Export

- Build `DatasetWriter` — SQLite schema, COCO JSON export, DVC versioning
- Build `AnnotationRouter` — auto-accept path + Label Studio push path
- Review ~200 flagged frames in Label Studio (you + Arunmozhi, 1hr each)
- First DVC commit — dataset v20250414_Nframes
- Target: 5,000 auto-accepted + 200 human-verified frames in dataset

WEEK 5

IMU Fusion + OpenMarcie

Processing

- OpenMarcie dataset should be approved — download 40GB
- Build `ComplementaryFilter` and `IMUAutoLabeler`
- Run IMU fusion on OpenMarcie wearable streams — validate fall detection fires correctly
- Process OpenMarcie video with skeleton extractor — adds industrial egocentric data to dataset
- Build feature extractor — 20-dim vector, test on processed frames

WEEK 6

Industrial Fine-Tune (YOLOv8-Pose)

- Fine-tune YOLOv8s-pose on SH17 + Roboflow on Vast.ai RTX 4090 ($~5 total)
- Log all metrics to MLflow. Compare mAP before/after on held-out SH17 val set.
- Process iSafetyBench 1,100 video clips with

pipeline — extract skeleton sequences

- Build action classifier training data from NTU RGB+D (if approved) + iSafetyBench labels

WEEK 7

Action Classifier + Risk Scorer

- Train ST-GCN action classifier on Colab Pro — target Top-1 > 78% on hazard classes
- Train XGBoost risk scorer on feature vectors — calibrate with isotonic regression
- Compute SHAP values — verify top features make physical sense (PPE score, zone proximity)
- Run end-to-end pipeline on one iSafetyBench video — check risk scores on hazardous vs normal clips

WEEK 8

Demo Build + Metrics + Traction Package

- Build `scripts/run_pipeline.py` — single entry point, any video in → annotated output
- Record demo video: input factory clip → skeleton overlay → risk scores → COCO JSON export
- Write 1-page metrics report: dataset size, mAP delta, action accuracy, risk ECE, throughput
- Prepare outreach to 5 robotics companies: "Give us your video. We return labeled skeletons in 24hr."

13 Master Agent Instructions

The following is the complete instruction set for an AI coding agent (Claude, Cursor, etc.) tasked with building this pipeline. Hand this section to the agent

as the system prompt or first message.

AGENT SYSTEM PROMPT — TENTELLECT PIPELINE BUILD

You are a senior systems and ML engineer building the Tentellect skeleton AI pipeline. Your job is to implement this codebase from scratch, following this PRD exactly. Here are your operating rules:

Code standards: Python 3.11. Type hints on every function. Docstrings on every class and public method. No print statements — use Python logging with levels DEBUG/INFO/WARN All config in

`configs/pipeline.` never hardcoded. All file paths relative to project root via pathlib.Path.

Testing: Every module must have a corresponding test file in `tests/`. Use pytest. Tests must run in under 30 seconds total using small fixture data — never require a full dataset download to run tests. Use a 5-frame synthetic video fixture for ingestion tests. Use 10 pre-saved skeleton dicts for quality gate tests.

Error handling: Never let the pipeline crash silently. All exceptions must be caught, logged with full traceback, and

the frame/clip must be routed to a `data/processed/er` directory with the error reason in a sidecar JSON file. The pipeline must continue processing remaining frames after any single-frame failure.

Build order — follow exactly:

1. `scripts/download_` → 2. `scripts/validate_` → 3. `src/ingestion/pre` → 4. `src/ingestion/ing` → 5. `src/skeleton/extr` → 6. `src/annotation/qu` → 7. `src/imu/fusion.py` → 8. `src/annotation/an` → 9. `src/annotation/st` → 10. `src/features/extr` → 11. `src/training/trai`

→ 12. `src/training/trai`
→ 13. `src/training/trai`
→ 14. `scripts/run_pipel`

After each module: Run its test file. If tests fail, fix before moving to next module. Do not skip ahead.

Hardware assumption: Dev machine is CPU-only. Cloud inference (Vast.ai) is used for GPU workloads. Write all inference code with a `device: str = "cpu"` parameter so it runs locally for testing and on GPU in production.

When you are unsure about a design decision: Default to the simpler option. Write a TODO comment with your reasoning.

Do not ask for clarification — make a decision and document it.

Agent Task Checklist — Complete Build

#	MODULE
01	Dataset downloader
02	Dataset validator
03	Frame preprocessor
04	Data ingestor
05	Skeleton extractor
06	Quality gates
07	IMU fusion
08	Annotation router
09	Dataset storage
10	Feature extractor
11	Pose fine-tune

#	MODULE
12	**Action classifier**
13	**Risk scorer**
14	**Pipeline entrypoint**
15	**Review puller**
16	**Dataset version commit**

14 Success Metrics

These are the numbers you measure at end of Week 8. If you hit all of them, you have a demo worth showing. If you miss any, you understand exactly what to fix.

METRIC	TARGE
Dataset size	`> 2(` `fram`

METRIC	TARGE
Pose mAP50 — industrial	`> 0`
Pose mAP50 delta	`> +0` vs pretr
Action Top-1 accuracy	`> 78`
Risk score ECE	`< 0`
Pipeline throughput	`> 50` `fram`
Auto-accept rate	`> 70`
Human review time	`< 10` `of v`

METRIC	TARGE

15 Known Risks & Mitigations

RISK	L
NTU RGB+D / Human3.6M request takes > 2 weeks	
Auto-accept rate below 70% on SH17	
Vast.ai GPU spot instance preempted mid-training	
ST-GCN action classifier underperforms on industrial video	
OpenMarcie IMU format incompatible with fusion layer	

RISK	L
No factory video to demo with publicly	

HARD STOP

Do not begin fine-tuning (Week 6) until the dataset has at least **5,000 auto-accepted frames from SH17** with quality gate passing. Training on low-confidence auto-annotations produces a model that is worse than the pretrained baseline — this is the most common mistake in fast-moving annotation pipelines.

Tentellect Internal — Confidential
PRD v1.0 · Authored by Anvin · April 2025
Next review: end of Week 4

TENTELLECT